

Runtime Enforcement of Architectural Irreversibility in Edge AI Systems

{This research was conducted at the Department of Computer Science and Engineering, Swarnandhra College of Engineering & Technology (Autonomous), as part of academic research activities.

Dr. S. Suresh Kumar / ScholarMaster Research Group
Swarnandhra College of Engineering & Technology (Autonomous)

Abstract—Architectural privacy claims require runtime validation to ensure compliance under adverse deployment conditions. This paper presents a verification methodology for confirming that privacy invariants—defined in ScholarMasterEngine’s canonical architecture—are enforced at runtime, including under crash and failure conditions. We enumerate a failure taxonomy covering process crashes, power loss, kernel panics, network partitions, clock skew, and watchdog failures. We describe the runtime enforcement mechanisms: volatile-memory-only processing, time-to-live (TTL) enforcement, forced zeroization, independent watchdog processes, and fail-closed semantics. We define formal irreversibility invariants and present a fault injection test harness that simulates failures at each pipeline stage. Results demonstrate zero application-layer post-crash data residue (DRAM-level remanence is outside the scope of this software verification), consistent watchdog termination success, and correct system halt behavior across all tested failure scenarios. This paper empirically verifies the runtime enforcement of the architectural claims defined in Paper 17, without modifying the underlying architecture.

Keywords—Runtime Privacy Verification, Crash-Oriented Enforcement, Temporal-Bound Supervision, Compiler-Safe Zeroization, Fault-Injection Validation, Watchdog Isolation.

\begin{abstract}

Architectural privacy claims require runtime validation to ensure compliance under adverse deployment conditions. This paper presents a verification methodology for confirming that privacy invariants—defined in ScholarMasterEngine’s canonical architecture—are enforced at runtime, including under crash and failure conditions. We enumerate a failure taxonomy covering process crashes, power loss, kernel panics, network partitions, clock skew, and watchdog failures. We describe the runtime enforcement mechanisms: volatile-memory-only processing, time-to-live (TTL) enforcement, forced zeroization, independent watchdog processes, and fail-closed semantics. We define formal irreversibility invariants and present a fault injection test harness that simulates failures at each pipeline stage. Results demonstrate zero application-layer post-crash data residue (DRAM-level remanence is outside the scope of this software verification), consistent watchdog termination success, and correct system halt behavior across all tested failure scenarios. This paper empirically verifies the runtime enforcement of the architectural claims defined in Paper 17, without modifying the underlying architecture.

\end{abstract}

\end{IEEEkeywords}

Runtime Privacy Verification, Crash-Oriented Enforcement, Temporal-Bound Supervision, Compiler-Safe Zeroization, Fault-Injection Validation, Watchdog Isolation.

\end{IEEEkeywords}

INTRODUCTION

Why Architectural Claims Require Runtime Verification

The dominant thesis of this paper is that architectural privacy claims require empirical runtime enforcement validation. The gap between policy and enforcement is the gap between design intent and observable behavior. Architectural privacy guarantees can easily fail operationally. Crash conditions frequently bypass intended invariants, and memory-management behavior under stress often diverges from theoretical design. Formal design principles alone do not guarantee enforcement; runtime failure semantics materially affect privacy. Therefore, empirical validation is required for deployment credibility.

Scope and Authority

This subsystem validates runtime privacy enforcement, verifies fail-closed operational behavior, tests crash resilience, evaluates TTL enforcement correctness, and empirically confirms runtime invariant preservation. Within the ScholarMaster architecture, this subsystem performs runtime enforcement validation only. It does not define formal threat models or TCB assumptions (Paper 19), derive spatiotemporal logic theorems (Paper 21), define federated optimization (Paper 14), or synthesize deployment architecture (Paper 20).

Our contribution is strictly empirical proof of enforcement. All invariants defined in this paper use the P18-INV namespace (P18-INV-01 through P18-INV-06) to distinguish them from the canonical invariant namespace.

Paper Structure

Section II defines the threat model and failure taxonomy. Section III describes runtime enforcement mechanisms. Section IV states formal irreversibility invariants. Section V presents the fault injection test harness. Section VI reports results. Section VII discusses implications. Section VIII clarifies the relationship to prior work. Section IX concludes.

THREAT MODEL AND FAILURE TAXONOMY

Threat Model

This threat model fits deploying edge appliances for use within managed institutional settings. The adversary wishes to exfiltrate raw sensor data (RGB frames, audio waveforms) which should have been shredded. The adversary could leverage:

- Timing windows during normal operation
- Memory residue after process termination
- Disk artifacts from swap or crash dumps
- Network captures during partition recovery

Failure Taxonomy: Operational Necessity

Privacy systems often exhibit failures during the abnormal state. Crash conditions are rarely considered in edge AI literature, where the majority of the evaluations are performed in an ideal scenario. The verification process under faulty-state assumptions is significantly different from the standard steady-state benchmarking. As such, outlining the failure taxonomy is operationally necessary to assess the privacy guarantees:

TABLE: Failure Taxonomy

Failure Tolerance Requirement

The system must maintain privacy invariants under all enumerated failures. Privacy degradation under any failure mode invalidates the architectural claim. There is no acceptable partial compliance.

RUNTIME ENFORCEMENT MECHANISMS

Volatile-Memory-Only Processing

Raw sensor data is processed exclusively in volatile memory. The system enforces this through:

- RAM disk allocation for sensor buffers
- Disabled swap partition (kernel parameter `vm.swappiness=0`)
- Crash dump exclusion via `/proc/$PID/coredump_filter`
- No file handles opened for raw data paths

Kernel-Level Memory Locking

To prevent the OS from paging raw data to persistent swap partitions (a common privacy leak), we enforce `mlock` on all sensor buffers. These mechanisms assume `CAP_IPC_LOCK` privileges, which are standard for edge system daemons. System startup verifies `CAP_IPC_LOCK`; failure prevents service launch.

```
\begin{lstlisting}[language=C, caption={Kernel Locking Strategy}]
```

```
// Enforce RAM-only residency
if (mlock(buffer_ptr, size) != 0) {
    perror("mlock failed - Security Risk");
    abort(); // Fail-closed
}
```

```
// Mark as MADV_DONTDUMP to exclude from cores
madvise(buffer_ptr, size, MADV_DONTDUMP);
\end{lstlisting}
```

This ensures that even if the kernel panics and dumps memory to disk for debugging, the raw sensor buffers are excluded from the dump file, satisfying P18-INV-01 under catastrophic failure [Ref].

Time-to-Live (TTL) Enforcement

Raw data buffers are tagged with creation timestamps using the system's monotonic clock (`CLOCK_MONOTONIC`) to neutralize NTP manipulation attacks. The TTL enforcement loop executes every

10ms:

```
\begin{lstlisting}[language=Python, caption={TTL Enforcement Loop}]
```

```
while running:
    now = monotonic_ns()
    for buffer in raw_buffers:
        age_ms = (now - buffer.created) / 1e6
        if age_ms > FRAME_TTL_MS: # 33ms
            zeroize(buffer)
            deallocate(buffer)
            log_ttl_expiry(buffer.id)
            sleep_ms(10)
\end{lstlisting}
```

The TTL constants are:

- `FRAME_TTL_MS` = 33 (video frames)
- `AUDIO_TTL_MS` = 3000 (audio buffers)

The TTL thresholds mentioned in the previous paragraph are calculated with respect to the frame cadence (30 FPS) and the bounded buffering constraints described in Paper 17; they must be treated as hard deadlines rather than average destruction times, as the implementation of the TTL enforcement thread runs in `SCHED_FIFO` priority domain with inference threads. The enforcement thread has deterministic execution bounds (jitter <2ms) measured on a CPU with all cores saturated (100% utilization).

Forced Zeroization

Before deallocation, all raw buffers are explicitly zeroized using compiler-safe techniques to prevent dead-store elimination optimization. Zeroization strictly removes application-layer recoverability; DRAM remanence forensics are outside scope.

```
\begin{lstlisting}[language=C, caption={Zeroization Procedure}]
void zeroize(buffer_t *buffer) {
    // Volatile cast prevents compiler optimization
    volatile uint8_t *vptr = (volatile uint8_t *)buffer->ptr;
    for(size_t i = 0; i < buffer->size; i++) {
        vptr[i] = 0;
    }
}
```

```
memory_barrier(); // Prevent reordering
verify_zero(buffer->ptr, buffer->size);
}
\end{lstlisting}
```

The `verify_zero` check forces a read-back to improve operational confidence, ensuring the write completed before deallocation. Failure to verify triggers an immediate halt.

Independent Watchdog Process

A separate watchdog process monitors TTL compliance, ensuring deterministic fail-closed escalation. The watchdog:

- Runs as a distinct process with separate memory space
- Executes at an equal or higher scheduling class (SCHED_FIFO) to enforce starvation resistance
- Queries the main process via shared memory fence heartbeat semantics

- Terminates the main process via SIGKILL if a TTL violation is detected
- Cannot be disabled by the main process

We note that watchdog independence remains OS-bound; scheduler correctness remains assumed, and severe kernel-level starvation remains outside these strict empirical guarantees.

[FIGURE: Architectural Pipeline / Validation Curves]

\caption{Watchdog Architecture. The Watchdog operates with process-level isolation within the same OS instance, monitoring the Shared Fence. If the Main Process hangs or violates TTL, the Watchdog issues a SIGKILL.}

```
\label{fig:watchdog}
\end{figure}
```

Kill-on-Violation Semantics

The system employs kill-on-violation semantics:

- TTL exceeded $\rightarrow$ SIGKILL
- Zeroization failed $\rightarrow$ SIGKILL
- Governance gate unreachable $\rightarrow$ output blocked
- Watchdog heartbeat missed $\rightarrow$ system halt

Fail-Closed Behavior

The system is fail-closed, not fail-open. Under any unrecoverable error:

- All output ports are disabled
- Raw buffers are zeroized
- Process terminates with non-zero exit code
- Restart requires explicit operator acknowledgment

This contrasts with fail-open systems that continue operating with degraded privacy.

FORMAL IRREVERSIBILITY INVARIANTS

The invariants below are organized as a layered enforcement chain. This sequence provides progressively enforced runtime containment. We define the following paper-local invariants (P18-INV namespace) which must hold under failure conditions.

P18-INV-01: Boundary Non-Crossing

Invariant P18-INV-01 validates the irreversibility boundary as defined in Paper 17.

Formally: $\forall d \in \text{RawData}, \forall t: \text{accessible}(d, t) \rightarrow \text{is_pre_boundary}(d, t)$

Enforcement: Type system prevents raw data types from appearing in post-boundary function signatures. Runtime assertions verify payload types at the boundary transition.

P18-INV-02: TTL Compliance

No process holds raw data beyond the defined TTL.

Formally: $\forall d \in \text{RawData}, \forall t: \text{accessible}(d, t) \rightarrow \text{age}(d, t) < \text{TTL}(d)$

Enforcement: TTL enforcement loop (10ms interval) + independent watchdog (100ms interval). Dual enforcement provides defense in depth.

Proof Sketch for P18-INV-02 (TTL Safety)

We verify P18-INV-02 using Hoare Logic triples [Ref]. Let S be the state of a raw buffer.

Equation: $\{ \text{Alloc}(b) \wedge t = T_0 \} \quad P \quad \{ \text{Zero}(b) \wedge t < T_0 + \Delta \}$

Where Δ is the TTL limit.

The watchdog process W operates concurrently with the main process M .

If M halts, W detects condition $C_{\text{stale}} = (t_{\text{current}} - t_{\text{created}}) > \Delta$.

The safety property is enforced because W executes as an independent process with OS-level scheduling separation and executes at an equal or higher scheduling class than the monitored process. Thus, even if M enters an infinite loop, W preempts it to enforce the post-condition.

P18-INV-03: Restart Non-Replay

System restart does not replay buffered raw data.

Formally: $\text{restart}() \rightarrow \text{RawBuffers} = \text{emptyset}$

Enforcement: Volatile memory is assumed to be cleared upon power loss under standard hardware behavior; physical remanence attacks are outside the defined threat model. Process termination triggers explicit zeroization. Boot sequence verifies empty buffer state before accepting new data.

P18-INV-04: Governance Non-Bypass

Invariant P18-INV-04 validates the governance gate non-bypass requirement as defined in Paper 17.

Formally: $\forall o \in \text{Output}: \text{passed_governance}(o) = \text{true}$

Enforcement: The output module requires an approval token. Token is cryptographically signed by the governance process. Missing or invalid token blocks output.

P18-INV-05: Watchdog Independence

The watchdog process operates independently of the monitored process.

Formally: $\text{crash} \not\rightarrow \text{crash}(\text{watchdog})$

Enforcement: Separate process, separate memory space, process-level isolation within the same OS instance. Watchdog monitors main process health and takes action on non-response.

P18-INV-06: Zeroization Completeness

Deallocated buffers contain no residual application-layer data.

Formally: $\forall d \in \text{Buffers}: \text{deallocate}(d) \rightarrow \text{content}(d) = 0 \text{ immediately prior to release}$

Enforcement: Zeroization before deallocation with verification. Failure to verify triggers halt. Note: this invariant is scoped to application-layer memory; DRAM-level data remanence (e.g., cold-boot attacks) requires physical countermeasures outside the scope of this software verification.

FAILURE INJECTION AND TEST HARNESS

Fault Injection Methodology

We employ systematic fault injection to verify invariant preservation under failure. All fault-injection scripts and invariant checkers are available in the ScholarMasterEngine repository under /tests/runtime_enforcement/. The test harness:

- Initializes the system with synthetic sensor data
- Injects a failure at a specified pipeline stage
- Waits for system response (halt, recovery, or timeout)
- Inspects application-layer memory, disk, and network for residue
- Verifies invariants held throughout

Failure Scenarios

TABLE: Fault Injection Scenarios

Note that the power loss scenario (F02) models OS-level sudden termination, not physical DRAM retention.

Residue Detection

Post-failure residue detection operates at the application layer and includes:

- Process memory scan for raw data patterns (frame headers, audio signatures)
- Disk scan for unexpected files in /tmp, swap, crash dumps
- Network capture analysis for leaked payloads
- Process memory dump inspection (where permitted by kernel)

Note: these checks verify the absence of recoverable application-layer references. Physical DRAM-level forensics (e.g., cold-boot extraction) are outside the scope of this software-level verification.

Test Harness Implementation

The test harness is implemented in Python with C extensions for memory inspection:

```
\begin{lstlisting}[language=Python, caption={Fault Injection Test Structure}]
class TestFailureScenario(unittest.TestCase):
    def setUp(self):
        self.system = start_system()
        self.inject_synthetic_data()
    def test_process_crash_no_residue(self):
        kill(self.system.pid, SIGKILL)
        wait_for_termination()
        residue = scan_memory_for_raw_data()
        self.assertEqual(residue, 0)
    def test_watchdog_kills_on_ttl_violation(self):
        pause_ttl_enforcement()
        sleep_ms(FRAME_TTL_MS + 100)
        self.assertFalse(is_running(self.system.pid))
\end{lstlisting}
```

Chaos Engineering Implementation

We utilized the Linux Kernel Fault Injection framework to perform synthetic failure orchestration and bounded fault-space exploration, following principles established by Natella et al. [Ref]. Specifically, we enabled:

- fail_make_request: To simulate partial disk write failures during logging.
- fail_alloc_page: To simulate OOM conditions in the middle of inference.
- fail_futex: To simulate locking deadlocks between the main process and watchdog.

The test harness is run for 24 hrs ("Soak Testing") to observe runtime resilience. With no invariant violations found across 14000+ injected failures under these test conditions, and noting that the limitations of laboratory test conditions can rarely encompass all possible failure modes, confidence is high that the software is operating safely under normal conditions.

RESULTS

TTL Enforcement

TABLE: TTL Enforcement Results

Watchdog Behavior

TABLE: Watchdog Termination Results

Post-Crash Application-Layer Data Residue

TABLE: Post-Crash Residue Detection (Application Layer)

System Halt Correctness

TABLE: System Halt Behavior

Invariant Verification Summary

TABLE: Invariant Verification

DISCUSSION

Empirical Validation of Fail-Closed Semantics

We empirically validate that fail-closed semantics trigger correctly under invariant violation. When a privacy invariant cannot be verified, the system reliably transitions to a safe HALT state.

[FIGURE: Architectural Pipeline / Validation Curves]

\caption{Fail-Closed State Machine. Any critical invariant violation (TTL, Watchdog) causes transition to the HALT state. The only possible transition caused by network problems is the move to the Degraded (Safe) state, from which no outputs are produced, but the local processing is still allowed.}

\label{fig:states}
\end{figure}

Adversarial Timing Analysis (TOCTOU)

We checked the system for Time-of-Check-to-Time-of-Use (TOCTOU) vulnerabilities during the destruction window. An attacker could possibly read the memory contents directly after the TTL check and before zeroization. However, the `zeroize()` function is executed atomically with respect to the memory allocator. The pointer is not released until `memset` completes. Moreover, the memory is mapped as `private` and `anonymous`, so that no other process can observe it. The only viable attack scenario we can think of would require a cold-boot physical memory dump of the server during the 33ms window, which is outside the scope of our threat model.

Defense in Depth

The enforcement mechanisms provide defense in depth:

- Primary: TTL enforcement loop (10ms)
- Secondary: Independent watchdog (100ms)
- Tertiary: Forced zeroization on deallocation
- Quaternary: Boot verification of empty state

Each layer can fail independently without compromising the overall enforcement, provided at least one layer remains operational.

Limitations and Operational Realism

The empirical verification of runtime privacy enforcement contains several methodological and structural boundaries. Operational reality involves severe resource exhaustion, scheduler contention, and institutional deployment instability, limiting our guarantees:

- Hardware Remanence Vulnerability: The software-only verification methodology cannot account for physical DRAM-level data remanence (e.g., cold-boot attacks) which require hardware-level countermeasures.
- Hypervisor Opacity: The verification assumes the operating system acts as the lowest layer of execution; hypervisor-level memory inspection by a malicious virtual machine monitor remains undetected.
- Scheduler Starvation Risk: Watchdog enforcement relies heavily on `SCHED_FIFO` priority; severe kernel-level CPU starvation or locking contention could hypothetically delay `SIGKILL` delivery beyond the TTL limit.
- Microarchitectural Leakage: While compiler-safe zeroization clears application memory, transient execution side-channels (e.g., Spectre/Meltdown variants) occurring within the 33ms window are not verified by this framework.
- Laboratory Synthetic Bias: Fault injection inherently tests anticipated failure distributions. Long-duration operational drift and novel, compound real-world crash behaviors fall outside the generated injection envelope.

SCOPE AND RELATION TO PRIOR WORK

Relationship to Paper 17

Paper 17 [Ref] defines the canonical eight-layer architecture and states architectural irreversibility as a design principle. This paper validates, but does not extend, those claims. We provide runtime evidence that the architecture behaves as specified, strictly isolating empirical verification from theoretical design.

Related Work

Fault injection testing has a long history in systems research [Ref]. Watchdog-based enforcement is standard in safety-critical systems [Ref]. Memory zeroization techniques derive from secure deletion literature [Ref]. Fail-closed semantics are established in safety-critical computing [Ref]. Our contribution is applying these techniques specifically to verify privacy-by-architecture claims in edge AI systems.

Hardware Enclaves vs. Software Enforcement

We rely on software enforcement rather than hardware enclaves. Architectural justification is detailed in Paper 17. This paper evaluates runtime correctness only.

Future Direction: Formal Verification

While this work relies on runtime assertion (dynamic verification), future work will explore static verification using TLA+ (Temporal Logic of Actions). We aim to model the interaction between the Watchdog (W) and Main Process (M) as a state machine:

$$\text{Equation: } \square (\text{State})(M) = \text{Stale} \implies \Diamond (\text{State})(M) = \text{Halted}$$

Proving this liveness property mathematically would complement the empirical evidence presented here, providing a complete verification stack from logic to runtime.

CONCLUSION

This runtime-verification study demonstrates that architectural privacy claims can be empirically bounded under failure conditions. Through systematic fault injection across 475 failure scenarios, we verified that within the evaluated runtime envelope, no recoverable application-layer post-crash data residue was observed. We confirmed consistent watchdog termination success and correct fail-closed system halt behavior. For all six paper-local invariants—enforcing boundary isolation, temporal exposure, restart protection, governance checking, supervisory independence, and memory sanitization—no violations were observed under tested conditions. The system guarantees that it halts rather than degrades privacy. Within the ScholarMaster architecture, this framework operates exclusively as the runtime verification and crash-oriented enforcement subsystem—validating temporal isolation and memory destruction without governing spatial visualization, cryptographic provenance, formal threat modeling, or distributed orchestration logic.

REFERENCES

- [1] C. A. R. Hoare, "An axiomatic basis for computer programming," *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [2] L. Lamport, "Proving the correctness of multiprocess programs," *IEEE Trans. Software Engineering*, vol. 3, no. 2, pp. 125–143, 1977. % Runtime Verification
- [3] J. Arlat et al., "Fault injection for dependability validation: A methodology and some applications," *IEEE Trans. Software Engineering*, vol. 16, no. 2, pp. 166–182, 1990.
- [4] N. G. Leveson, *Safeware: System Safety and Computers*. Addison-Wesley, 1995. % Privacy by Design
- [5] P. Gutmann, "Secure deletion of data from magnetic and solid-state memory," *USENIX Security*, 1996. % Fail-Safe Systems
- [6] M. Hsueh, T. Tsai, and R. Iyer, "Fault injection techniques and tools," *Computer*, vol. 30, no. 4, pp. 75–82, 1997. % Watchdog Systems
- [7] A. Benso et al., "A Watchdog Processor to Detect Data and Control Flow Errors," *IEEE Design & Test of Computers*, vol. 15, no. 4, pp. 24–32, 1998. % Secure Deletion
- [8] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. MIT Press, 1999.
- [9] J. Rushby, "Partitioning in avionics architectures: Requirements, mechanisms, and assurance," *NASA Contractor Report*, 1999.
- [10] M. Leucker and C. Schallhart, "A brief account of runtime verification," *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009. % New References
- [11] A. Cavoukian, "Privacy by design: The 7 foundational principles," *Information and Privacy Commissioner of Ontario*, 2009. % Edge AI
- [12] L. Szekeres et al., "SoK: Eternal war in memory," *IEEE S&P*, 2013. % Differential Privacy (Referenced in Context)
- [13] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2014.
- [14] C. Dwork et al., "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical CS*, 2014. % ScholarMaster Series
- [15] R. Natella et al., "Assessing dependability with software fault injection: A survey," *ACM Computing Surveys*, vol. 48, no. 3, pp. 1–27, 2016.
- [16] V. Costan and S. Devadas, "Intel SGX Explained," *IACR Cryptology ePrint Archive*, 2016.
- [17] Y. Mao et al., "A survey on mobile edge computing," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017. % Memory Safety
- [18] N. Babu P., "Privacy-preserving pose analytics via architectural irreversibility," *ScholarMaster Series*, Paper 3, 2025.
- [19] N. Babu P., "Mandatory governance gates for edge AI systems," *ScholarMaster Series*, Paper 10, 2025.
- [20] N. Babu P., "Visible privacy indicators in intelligent environments," *ScholarMaster Series*, Paper 11, 2025.
- [21] N. Babu P., "Fail-safe semantics for privacy-critical edge systems," *ScholarMaster Series*, Paper 9, 2025.
- [22] N. Babu P., "Architectural irreversibility: Enforcing privacy, governance, and trust in intelligent campus systems," *ScholarMaster Series*, Paper 17 (Capstone), 2026.
- [23] S. Suresh Kumar, "Perception Integrity Foundations: Evidential Uncertainty and Calibrated Disagreement in Edge Vision," *ScholarMaster Research Series*, Paper 22, 2026.
- [24] S. Suresh Kumar, "ScholarMaster Integration Architecture and Downstream Error Propagation Analysis," *ScholarMaster Research Series*, Paper 25, 2026.
- [25] S.~Suresh~Kumar, "Perception integrity foundations: Evidential uncertainty, disagreement dynamics, and blur bounds in edge vision," *\emph{ScholarMaster Technical Report Series}*, Paper 22, 2026.
- [26] S.~Suresh~Kumar, "Adaptive trustworthy edge systems: Dynamic risk-driven cascades and real-time SLA bounds," *\emph{ScholarMaster Technical Report Series}*, Paper 23, 2026.